

Code Explanation

Testing Transformations Code Explanation

Overview

The given code snippet is a part of a test suite written in Python using the Pytest framework. It is designed to test various data transformation functions using Apache Spark. The code defines a test fixture for creating a Spark session and imports necessary transformation functions from the `src.transformations` module.

Step 1: Importing Necessary Modules and Defining Test Fixture

The first step involves importing the required modules, including Pytest and PySpark. It also defines a test fixture named `spark` using the `@pytest.fixture` decorator. This fixture is used to create a Spark session for testing purposes.

```
import pytest
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, TimestampType
from datetime import datetime
```

Step 2: Importing Transformation Functions

The next step is to import the transformation functions that are being tested. These functions are imported from the `src.transformations` module.

```
from src.transformations import (
    check_source_record_count, format_reception_date, normalize_postal_code,
    apply_address_fallback, format_telephone_number, format_policy_owner_name,
    add_sequence_number
)
```

Step 3: Defining the Spark Test Fixture

The `spark` test fixture is defined using the `@pytest.fixture` decorator with a scope of "module". This means that the Spark session will be created once for the entire test module.

```
@pytest.fixture(scope="module")
def spark():
    """Create a Spark session for testing."""
    return SparkSession.builder
```

However, the SparkSession is not fully created as the `getOrCreate()` method is missing. The correct code should be:

```
@pytest.fixture(scope="module")
def spark():
    """Create a Spark session for testing."""
```

```
return SparkSession.builder.getOrCreate()
```